Eigen-Séta v2 in Rust

Constant-time decapsulation at NIST level 1, 3 and 5 parameter sizes: results report

2026-10-11 · code: eigen_seta/ (commit 19c641d12 for all measurements) · runs: run-20261011T1813Z-bench, run-20261011T1827Z-sidechannel

Claims and status

#	Claim	Status	Evidence
1	The Rust KEM decapsulates correctly at Toy16, Toy32, L1, L3 and L5.	measured, 15/15 trials	bench run
2	pk = ct = 1344 / 1992 / 2632 B at L1 / L3 / L5. Decaps takes 13–15 / 52–54 / 120–124 s on one core of the recorded host.	measured, n = 3 / 2 / 2, one host	bench run
3	The decaps secret phase runs a key- and ciphertext-independent instruction stream, apart from glibc's allocator.	measured at Toy16, 4 keys × 3 cts. Evidence, not proof.	side-channel run
4	No timing difference between a fixed ct and fresh ones, FO-rejected included, for one key.	measured, Welch $\|t\| \le 0.84$, n = 1,000 per class	side-channel run
5	The parameters target λ = 128 / 192 / 256 classical bits.	**conjectured**, heuristic. Quantum security ≈ λ/2, as for FESTA.	eigen_seta/ README.md §3
6	The toy `walk` variant in the merged prototype is insecure.	measured: an outsider decrypted 5/5 ciphertexts	correction notices

Two defects found during this work are fixed in the same PR. The first is an LLVM-introduced conditional in the safegcd normalisation, which skipped a masked add when the mask was zero. That leaked the sign of a secret-dependent intermediate, and the callgrind probe found it. The second is a key-generation hang: Tonelli–Shanks accepted z = 0 as a non-residue.

Correctness, sizes and speed

Run run-20261011T1813Z-bench, Intel Xeon at 2.80 GHz (shared VM), rustc 1.97.0. Every trial's decapsulated key matched.

Set	p bits	pk = ct (B)	keygen (s), n=1	encaps (s)	decaps (s)	correct
Toy16	216	216	0.38	0.005–0.008	0.12–0.19	5/5
Toy32	369	376	2.3	0.009–0.012	0.45–0.59	3/3
L1	1341	1344	32.3	0.24–0.26	13.4–15.0	3/3
L3	1991	1992	118.6	1.03	51.8–53.9	2/2
L5	2630	2632	271.8	2.17–2.20	120.2–124.0	2/2

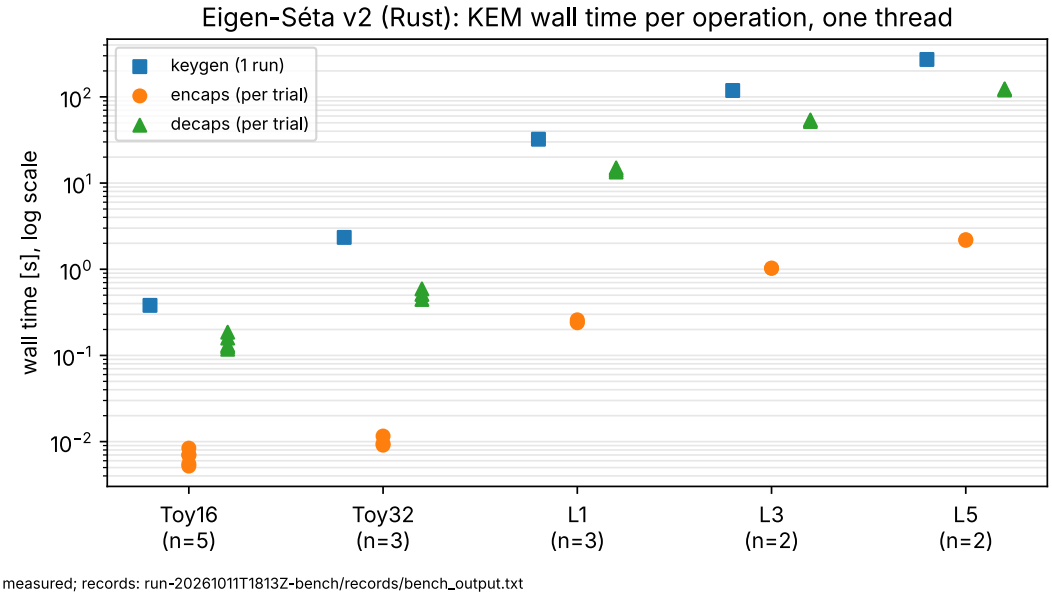
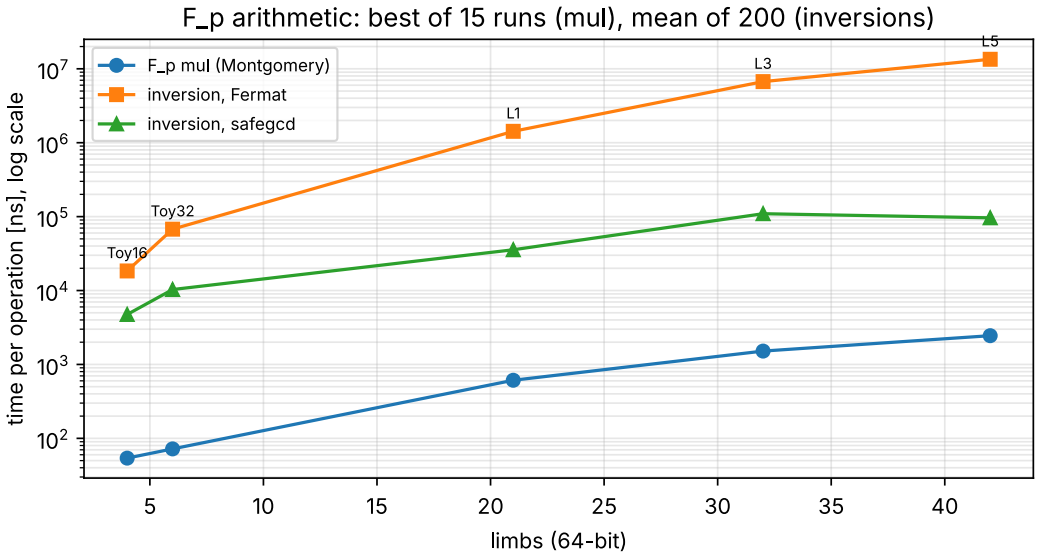


Figure 1: Wall time per operation, all trials, log scale. Measured; run-20261011T1813Z-bench/records/bench_output.txt.

In the same binary and run, safegcd inversion is 3.9× to 140× faster than Fermat inversion, from Toy16 to L5. No pre-change baseline binary was kept for the KEM-level timings, so this report makes **no KEM-level speedup claim**; AGENTS.md §10

requires a recorded baseline. Decapsulation time is dominated by the Richelot steps in Mumford coordinates, according to a callgrind profile at Toy16. A theta-coordinate chain is the main open performance item, and its gain is unmeasured.

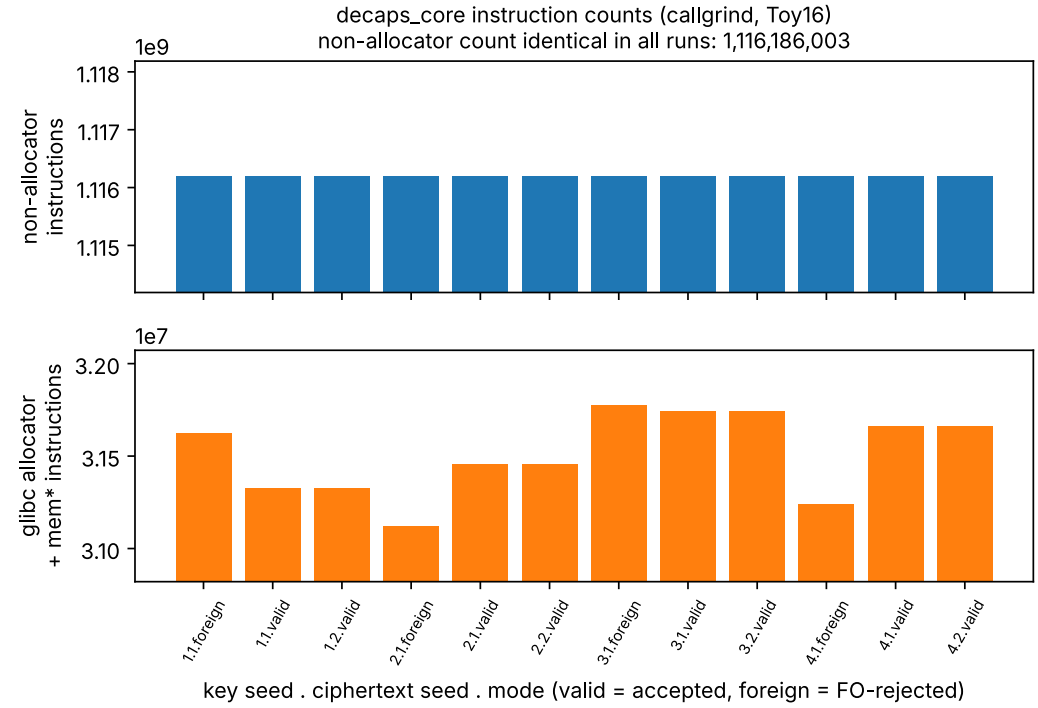


measured; records: run-20261011T1813Z-bench/records/bench_output.txt

Figure 2: F_p multiplication and the two inversions against limb count. Measured; same record.

Constant-time evidence

Run run-20261011T1827Z-sidechannel counts the instructions executed by `decaps_core`, the secret phase, under callgrind at Toy16. Each of 4 key seeds has 2 valid ciphertexts and 1 FO-rejected one. The non-allocator count was 1,116,186,003 in all 12 runs. The glibc allocator count varies with the heap state left by the earlier variable-time phases, and it is equal for the two valid ciphertexts of each key.



measured; records: run-20261011T1827Z-sidechannel/records/ctprobe_summary.txt

Figure 3: Instruction counts per run. Measured; run-20261011T1827Z-sidechannel/records/ctprobe_summary.txt.

Limits:

- Instruction counts do not cover memory-address leakage. The code has no secret-indexed tables, which is an argument, not a measurement.
- The check ran at Toy16 only. The code is generic over the limb count, but L1–L5 were not measured.

- The result covers this compiler and target only.
- Key generation is variable time by design (SIDE_CHANNELS.md, R1).

Graphs checked and left unchanged

These canonical graphs were checked: docs/index-calculus-scoreboard.html, docs/ic/progress-timeline.json, docs/performance-gains/, docs/curves/ and figures/. None covers isogeny KEMs. docs/curves/ mentions isogenies only as curve-transfer links in the discrete-log registry, and this work adds no registry curve. This report's graphs are new and are drawn from the run records by figures/plot.py, which only plots recorded values.